

Types of Function Invoking



Understanding different ways to call functions in C programming



Function Invoking Categories

We categorize a function's invoking (calling) depending on arguments or parameters and their returning a value. In simple words we can divide a function's invoking into four types depending on whether parameters are passed to a function or not and whether a function returns some value or not.





No Arguments

No Return Value



No Arguments

With Return Value



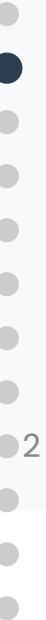
With Arguments

No Return Value



With Arguments

With Return Value



TYPE-1: With no arguments and have no return value

As the name suggests, any function which has no arguments and does not return any values to the calling function, falls in this category. These type of functions are confined to themselves i.e.

neither do they receive any data from the calling function nor do they transfer any data to the calling function. So there is no data communication between the calling and the called function are only program control will be transferred.



Example

```
/* Program for illustration of the function with no arguments and no return value*/
/* Function with no arguments and no return value*/
#include<stdio.h>
main()
{
    void message();
    printf("Control is in main\n");
    message(); /* Type 1 Function */
    printf("Control is again in main\n");
}
void message()
{
    printf("Control is in message function\n");
} /* does not return anything */
```

TYPE-1: Output



Output

```
Control is in main  
Control is in message function  
Control is again in main
```

Explanation



Control Flow

The main function calls `message()` without passing any arguments



No Data Transfer

No data is exchanged between `main()` and `message()`



No Return Value

message() doesn't return any value to main()



TYPE-2: With no arguments and with return value

Suppose if a function does not receive any data from calling function but does send some value to the calling function, then it falls in this category.



Example: Sum of first ten natural numbers

```
/* Program to find sum of first ten natural numbers */  
#include<stdio.h>
```

```
int cal_sum()
{
    int i, s=0;
    for (i=0; i<=10; i++) s=s + i;
    return(s); /* function returning sum of first ten natural numbers */
}
main()
{
    int sum;
    sum = cal_sum();
    printf("Sum of first ten natural numbers is %d\n", sum);
}
```

TYPE-2: Output



Output

Sum of first ten natural numbers is 55

Explanation



No Arguments

`cal_sum()` is called without any arguments



Returns Value

`cal_sum()` returns the sum (55) to `main()`



Data Storage

The returned value is stored in the `sum` variable

TYPE-3: With arguments and have no return value

If a function includes arguments but does not return anything, it falls in this category. One way communication takes place between the calling and the called function.

Types of Arguments

1 2
3 4

Actual Arguments

Arguments that are passed to the function when it is called



Formal Arguments

Arguments that are received by the function definition

TYPE-3: Example



Calculate sum of any three given numbers

```
/* Program to calculate sum of any three given numbers */
#include<stdio.h>
main()
{
    int a1, a2, a3;
    void sum(int, int, int);
    printf("Enter three numbers: ");
    scanf ("%d%d%d",&a1,&a2,&a3);
    sum (a1,a2,a3); /* Type 3 function */
}
/* function to calculate sum of three numbers */
void sum (int f1, int f2, int f3)
{
    int s;
    s = f1+ f2+ f3;
    printf ("\nThe sum of the three numbers is %d\n",s);
}
```

TYPE-3: Output and Explanation



Output

```
Enter three numbers: 23 34 45
The sum of the three numbers is 102
```

Explanation



Arguments Passed

a1, a2, a3 (actual arguments) are passed to sum() function



Arguments Received

f1, f2, f3 (formal arguments) receive the values



No Return Value

sum() doesn't return any value to main()

Here f1, f2, f3 are **formal arguments** and a1, a2, a3 are **actual arguments** .

9

TYPE-4: With arguments function and with return value

In this category two-way communication takes place between the calling and called function i.e. a function returns a value and also arguments are passed to it.



Example: Calculate sum of three numbers

```
/*Program to calculate the sum of three numbers*/  
#include<stdio.h>  
main( )  
{  
    int a1, a2, a3, result;
```

```
int sum(int, int, int);
printf("Please enter any 3 numbers:\n");
scanf ("%d %d %d", &a1, &a2, &a3);
result = sum (a1,a2,a3); /* function call */
printf ("Sum of the given numbers is : %d\n", result);
}
/* Function to calculate the sum of three numbers */
int sum (int f1, int f2, int f3)
{ return(f1+ f2 + f3); /* function returns a value */
}
```

TYPE-4: Output



Output

```
Please enter any 3 numbers:
3 4 5
Sum of the given numbers is: 12
```

Explanation



Arguments Passed

a1, a2, a3 (actual arguments) are passed to sum() function



Arguments Received

f1, f2, f3 (formal arguments) receive the values



Returns Value

sum() returns the calculated sum to main()

Summary of Function Types

Type	Arguments	Return Value	Data Flow
TYPE-1	No	No	None

